

MODULE

4

List Functions

Master every List.* function in M — from creation and inspection through transformation, aggregation, set operations, and advanced iteration with List.Accumulate and List.Generate. Build dynamic, reusable query patterns that make you a professional-grade Power Query developer.

Module Contents

■ Module Overview

■ Lesson 1: Understanding the List Type

- What Is a List in M?
- The Relationship Between Lists, Records, and Tables
- List Indexing and the {n} Operator
- List Literal Syntax and Character Ranges
- Type Annotations for Lists

■ Lesson 2: Creating Lists

- List Literals: {} Syntax
- List.Dates, List.Times, List.Datetimes
- List.Numbers — Arithmetic Sequences
- List.Generate — Iterative List Construction
- Table Column to List: table[ColumnName]
- Record Fields to List: Record.FieldValues

■ Lesson 3: Inspecting and Accessing Lists

- List.Count — How Many Elements?
- List.First, List.Last, List.FirstN, List.LastN
- List.IsEmpty and Null-Safe Access
- List.Contains, List.ContainsAll, List.ContainsAny
- List.PositionOf and List.PositionOfAny
- List.Min, List.Max, List.Range
- List.Percentile, List.Median, List.Mode

■ Lesson 4: Transforming Lists

- List.Transform — Apply a Function to Every Element
- List.Select — Filter Elements by Condition
- List.Sort — Sort with Custom Comparers
- List.Distinct — Remove Duplicates
- List.Reverse, List.Skip, List.RemoveFirstN, List.RemoveLastN
- List.RemoveItems, List.RemoveMatchingItems, List.RemoveNulls
- List.Repeat, List.Zip
- List.ReplaceValue and List.ReplaceRange

■ Lesson 5: Aggregating Lists

- List.Sum, List.Average
- List.Min, List.Max and Custom Comparers

- List.StandardDeviation, List.Variance
- List.Covariance, List.Correlation
- List.Percentile and List.Median
- List.Product and List.Mode
- Building Custom Aggregations with List.Accumulate

■ Lesson 6: Set Operations and Combining Lists

- List.Union — Combine Without Duplicates
- List.Intersect — Common Elements
- List.Difference — Elements in One but Not the Other
- List.Combine — Concatenate Lists
- List.Zip — Pair Elements Across Lists
- Converting Lists: List.ToTable, Table.ToList

■ Lesson 7: Advanced List Patterns for PL-300

- List.Accumulate — Running Totals and State Machines
- List.Generate — Parameterised Sequences and Pagination
- Using Lists for Dynamic Column Selection
- Using Lists to Drive Table.SelectColumns and Table.AddColumn Dynamically
- Pattern: Lookup Without a Table Join
- Pattern: Multi-Value Parameters in Filters
- Pattern: Building a Cross-Join Table from Two Lists

■ Demonstration 1: Dynamic Column Selection and Reordering

■ Demonstration 2: Paginated API Ingestion with List.Generate

■ Lab 4: List Functions

- Exercise 1: List Creation, Inspection, and Indexing
- Exercise 2: List Transformation and Filtering
- Exercise 3: List Aggregation and Set Operations
- Exercise 4: Advanced Patterns — List.Accumulate and List.Generate

■ Module Review and Takeaways

- Review Questions
- Best Practices
- Common Issues and Solutions
- Key Functions Quick Reference

Module Overview

The **list** is the unsung hero of the M language. While most Power Query developers interact primarily with tables, lists are the engine running behind nearly every advanced technique: dynamic column selection, multi-value filtering, lookup operations without joins, pagination in API connectors, running totals, and iterative sequence generation. A developer who understands lists fluently can solve problems that are literally impossible to express using only the ribbon UI.

This module covers the complete **List.*** function library — from the fundamentals of creation and indexing through transformation, aggregation, set operations, and the two most powerful iterative functions in M: **List.Accumulate** and **List.Generate**. Each lesson builds on the previous, culminating in seven professional-grade patterns directly applicable to PL-300 exam scenarios and production models.

NOTE This module assumes familiarity with Modules 1-3. You should be comfortable with the `let...in` structure, `Table.AddColumn`, `Table.SelectRows`, the `each` keyword, and type annotations before proceeding.

Before You Begin

- Complete Modules 1-3 and their lab exercises.
- Have the FactSales and DimDate queries from previous labs loaded in Power BI Desktop.
- Download the lab file: Products_API_Sample.json from the companion materials (simulates a paginated REST API response).

LESSON 1

Understanding the List Type

After completing this lesson, you will be able to:

- Describe what a list is in M and how it differs from a table and a record.
- Access list elements by zero-based index using the `{n}` operator.
- Write list literals including mixed-type lists and character ranges.
- Explain how lists relate to table columns and record field values.
- Apply type annotations to list values.

What Is a List in M?

A **list** in M is an ordered, zero-indexed sequence of values. Unlike a record (which has named fields) or a table (which has named, typed columns and multiple rows), a list is a flat, positional sequence. It can hold any values of any type — numbers, text, dates, records, other lists, or a mix:

List literals in M:

```
// A list of numbers
Scores = {92, 74, 88, 61, 95}

// A list of text
Regions = {"East", "West", "Central", "South"}

// A mixed-type list (valid – M lists are not typed by element)
Mixed = {1, "hello", true, #date(2024, 3, 15), null}

// A list of records (this is what a table column of records looks like)
People = {[Name="Alice", Age=30], [Name="Bob", Age=25]}

// An empty list
Empty = {}

// A nested list (list of lists)
Matrix = {{1,2,3}, {4,5,6}, {7,8,9}}
```

The Relationship Between Lists, Records, and Tables

Understanding how these three structured types relate is fundamental. Every table is built from lists and records:

Type	Structure	Access Pattern	Analogy
List	Ordered sequence of values	{n} index	A column extracted from a table
Record	Named set of field-value pairs	[FieldName]	A single row of a table
Table	Ordered set of named, typed columns	[ColumnName] or {rowIndex}	A spreadsheet / database table

A table column accessed via **table[ColumnName]** returns a list. A table row accessed via **table{rowIndex}** returns a record. This symmetry is deliberate and deeply useful:

Accessing table data as lists and records:

```
// Given a table called Sales:
// | OrderID | Sales | Region |
// | CA-001 | 1200 | West |
// | CA-002 | 850 | East |

// Get the Sales column as a list:
SalesList = Sales[Sales] // = {1200, 850, ...}
```

```
// Get row 0 as a record:  
FirstRow = Sales{0} // = [OrderID="CA-001", Sales=1200, Region="West"]  
  
// Get a specific cell (row 0, Sales column):  
FirstSale = Sales{0}[Sales] // = 1200  
// or equivalently:  
FirstSale = Sales[Sales]{0} // = 1200
```

List Indexing

Lists are **zero-indexed**: the first element is at index {0}, the second at {1}, and so on. Attempting to access an index beyond the list length throws an error — use List.Count and conditional checks to guard against this:

```
Colors = {"Red", "Green", "Blue", "Yellow"}  
  
First = Colors{0} // = "Red"  
Third = Colors{2} // = "Blue"  
Last = Colors{List.Count(Colors) - 1} // = "Yellow" (safe last element)  
  
// Negative indexing is NOT supported – use List.Last() instead  
LastSafe = List.Last(Colors) // = "Yellow"  
  
// Out-of-bounds access throws an error:  
// Colors{99} -- Expression.Error: The index 99 is out of range  
  
// Safe access pattern with null default:  
SafeAccess = if List.Count(Colors) > 5 then Colors{5} else null  
// or use List.First with a default:  
Safe2 = List.First(List.Skip(Colors, 5), null) // null if fewer than 6 elements
```

Character Ranges in Lists

M supports a special range syntax inside list literals for consecutive characters and numbers. This is used most often with Text.Select and Text.Remove:

```
// Number range – generates {1, 2, 3, 4, 5}  
OneToFive = {1..5}  
  
// Character range – generates {"a", "b", "c", ..., "z"}  
LowerAlpha = {"a".. "z"}  
  
// Digits  
Digits = {"0".. "9"}  
  
// Combined – keep only letters and digits  
Alphanumeric = {"a".. "z"} & {"A".. "Z"} & {"0".. "9"}  
  
// Use with Text.Select to extract only alphanumeric characters:
```

```
Clean = Text.Select("Order #AB-123!", {"a".."z"} & {"A".."Z"} & {"0".."9"})  
// = "OrderAB123"
```

TIP The & operator concatenates two lists (equivalent to List.Combine). {"a".."z"} & {"A".."Z"} produces a single list of 52 characters. This is one of the few places in M where & works on non-text values.

LESSON 2

Creating Lists

After completing this lesson, you will be able to:

- Create numeric sequences with List.Numbers.
- Generate date, time, and datetime sequences with List.Dates, List.Times, List.DateTimes.
- Build iterative sequences with List.Generate.
- Extract a table column as a list using the table[column] syntax.
- Extract record field values as a list using Record.FieldValues.

List.Numbers — Arithmetic Sequences

List.Numbers

Syntax	List.Numbers(start as number, count as number, optional increment as nullable number) as list	
Parameter	Type	Description
start	<i>number</i>	The first value in the sequence.
count	<i>number</i>	Total number of values to generate.
increment	<i>nullable number</i>	Optional. Step between values. Default: 1.
Returns	Returns a list of numbers starting at start, with count elements, each separated by increment.	
Example	List.Numbers(0, 5, 10)	
Result	{0, 10, 20, 30, 40}	
NOTE	Use List.Numbers to generate row indices, percentage steps, or any arithmetic sequence. List.Numbers(1, 12, 1) generates {1..12} — useful for month number lists. List.Numbers(0, 1, 0.1) generates {0, 0.1, 0.2, ..., 0.9, 1.0} — a percentage scale.	

List.Dates

Syntax	List.Dates(start as date, count as number, step as duration) as list
---------------	--

Parameter	Type	Description
start	date	The first date in the sequence.
count	number	Number of dates to generate.
step	duration	Interval between dates. Use #duration(1,0,0,0) for daily, #duration(7,0,0,0) for weekly.
Returns	Returns a list of date values as a contiguous sequence.	
Example	List.Dates(#date(2024,1,1), 4, #duration(1,0,0,0))	
Result	{#date(2024,1,1), #date(2024,1,2), #date(2024,1,3), #date(2024,1,4)}	
NOTE This is the foundation of the Date dimension table from Module 3. Use step = #duration(7,0,0,0) for weekly sequences, or Date.AddMonths logic for monthly.		

List.Times	Generate a sequence of time values at regular intervals. Useful for shift schedule tables.
List.Times(start as time, count as number, step as duration) as list	List.Times(#time(9,0,0), 4, #duration(0,2,0,0)) → {#time(9,0,0), #time(11,0,0), #time(13,0,0), #time(15,0,0)}
List.DateTimeTimes	Generate a sequence of datetime values. Same pattern as List.Dates but with time component.
List.DateTimeTimes(start, count, step)	List.DateTimeTimes(#datetime(2024,1,1,0,0,0), 3, #duration(0,1,0,0)) → {#datetime(2024,1,1,0,0,0), #datetime(2024,1,1,1,0,0), #datetime(2024,1,1,2,0,0)}
List.DateTimeZones	Generate datetimezone sequences. Same pattern but preserves timezone offset.
List.DateTimeZones(start, count, step)	List.DateTimeZones(#datetimezone(2024,1,1,0,0,0,0,0), 2, #duration(1,0,0,0)) → {#datetimezone(2024,1,1,...), #datetimezone(2024,1,2,...)}

List.Generate — Iterative Construction

List.Generate is one of the most powerful functions in M. It constructs a list by iterating from an initial state, testing a condition, and applying a transformation at each step — similar to a while loop in imperative languages. It is the primary tool for building paginated API requests:

List.Generate	
Syntax	List.Generate(initial as function, condition as function, next as function, optional selector as nullable function) as list

Parameter	Type	Description
<code>initial</code>	<i>function</i>	<code>() => initialState</code> — A zero-argument function returning the starting state.
<code>condition</code>	<i>function</i>	<code>(state) => logical</code> — Returns true while the loop should continue.
<code>next</code>	<i>function</i>	<code>(state) => newState</code> — Transforms the current state into the next state.
<code>selector</code>	<i>nullable function</i>	Optional. <code>(state) => value</code> — Extracts the output value from each state. If omitted, the state itself is the output.

Returns	Returns a list by iterating: start with <code>initial()</code> , continue while <code>condition()</code> is true, advance with <code>next()</code> , output <code>selector()</code> at each step.
Example	<code>List.Generate(() => 1, each _ <= 5, each _ + 1)</code>
Result	<code>{1, 2, 3, 4, 5}</code>

NOTE Think of `List.Generate` as: `initial` = seed, `condition` = while loop test, `next` = loop body, `selector` = what to capture each iteration. The state can be any M value — a number, a record (for multi-variable state), or a list.

List.Generate with Record State — Fibonacci Sequence

Fibonacci with List.Generate:

```
// Generate first 10 Fibonacci numbers using a record for two-variable state
Fibonacci = List.Generate(
() => [a = 0, b = 1], // Initial state: a=0, b=1
each [a] < 100, // Continue while a < 100
each [a = [b], b = [a] + [b]], // Next state: swap and sum
each [a] // Output: the 'a' value each step
)
// = {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89}
```

List.Generate for API Pagination — the Key PL-300 Pattern

API pagination with List.Generate:

```
// Simulate paginated API: fetch pages until an empty page is returned
AllPages = List.Generate(
() => [Page = 1, Data = GetPage(1)],
// condition: keep going while last page had data
each [Data] <> null and Table.RowCount([Data]) > 0,
// next: increment page number and fetch next page
each [Page = [Page] + 1, Data = GetPage([Page] + 1)],
// selector: return only the data table from each state
each [Data]
)
// AllPages is now a list of tables, one per page
// Combine into a single table:
Combined = Table.Combine(AllPages)
```

```
// GetPage would be a custom function like:  
// GetPage = (pageNum as number) as nullable table => ...  
// (covered in full in Module 8: Custom Functions and APIs)
```

TIP List.Generate is covered again in Module 8 (Custom Functions and APIs) with a complete, production-ready paginated REST API connector. The pattern above is the conceptual foundation — understand initial/condition/next/selector and you will handle any pagination scenario.

Extracting a Table Column as a List

The most common way to create a list in real queries is by extracting a column from an existing table. This is used for lookups, set operations, and passing column values to list functions:

```
// table[ColumnName] returns the column as a list  
AllRegions = FactSales[Region] // list of all Region values (with duplicates)  
UniqueRegions = List.Distinct(FactSales[Region]) // deduplicated  
  
// Use in a filter – keep only rows where Region is in a valid list  
ValidRegions = {"East", "West", "Central", "South"}  
Filtered = Table.SelectRows(FactSales,  
each List.Contains(ValidRegions, [Region]))  
  
// Pass to List.Sum for a total  
TotalSales = List.Sum(FactSales[Sales])  
  
// The list approach avoids needing a separate aggregation step
```

LESSON 3

Inspecting and Accessing Lists

After completing this lesson, you will be able to:

- Count list elements with List.Count.
- Access first and last elements safely using List.First and List.Last.
- Retrieve subsets with List.FirstN and List.LastN.
- Test membership with List.Contains, List.ContainsAll, List.ContainsAny.
- Find element positions using List.PositionOf and List.PositionOfAny.
- Retrieve statistical summary values: Min, Max, Percentile, Median, Mode.

Counting and Boundary Access

List.Count

Syntax	List.Count(list as list) as number	
Parameter	Type	Description
list	list	The list to count.
Returns	Returns the number of elements in the list. An empty list returns 0.	
Example	List.Count({"a", "b", "c", "d"})	
Result	4	
NOTE	Always use List.Count before accessing by index to prevent out-of-bounds errors. List.Count also works on a table column: List.Count(table[Column]) = Table.RowCount(table).	

List.First

Syntax	List.First(list as list, optional defaultValue as any) as any	
Parameter	Type	Description
list	list	The source list.
defaultValue	any	Optional. Value to return if the list is empty. Default: null.
Returns	Returns the first element of the list, or defaultValue if the list is empty.	

Example	List.First({10, 20, 30}, 0)
Result	10

NOTE Always pass a defaultValue when the list might be empty. List.First({}, null) returns null — safe. List.First(Table.SelectRows(t, each [ID]=99)[Name], "Not Found") is a common lookup pattern.

List.Last		
Syntax	List.Last(list as list, optional defaultValue as any) as any	
Parameter	Type	Description
list	list	The source list.
defaultValue	any	Optional. Default if the list is empty.
Returns	Returns the last element of the list, or defaultValue if the list is empty.	
Example	List.Last({"Mon", "Tue", "Wed", "Thu", "Fri"}, null)	
Result	"Fri"	

NOTE Use List.Last rather than list{List.Count(list)-1} — it is both safer (handles empty lists) and more readable.

List.FirstN	Returns the first n elements. If countOrCondition is a function, returns elements while it is true.
List.FirstN(list, countOrCondition)	List.FirstN({1,2,3,4,5}, 3) → {1, 2, 3}
List.LastN	Returns the last n elements. If countOrCondition is a function, returns the last elements satisfying it.
List.LastN(list, countOrCondition)	List.LastN({1,2,3,4,5}, 2) → {4, 5}
List.IsEmpty	Returns true if the list has no elements. Use before List.First/Last to avoid unexpected nulls.
List.IsEmpty(list)	List.IsEmpty({}) → true
List.Range	Returns a sublist starting at offset (zero-based) with count elements. Equivalent to a list slice.
List.Range(list, offset, optional count)	List.Range({10,20,30,40,50}, 1, 3) → {20, 30, 40}

Membership Testing

Membership functions test whether specific values exist in a list. They are essential for validation, lookup, and filtering patterns:

List.Contains

Syntax	List.Contains(list as list, value as any, optional equationCriteria as any) as logical	
Parameter	Type	Description
list	list	The list to search.
value	any	The value to look for.
equationCriteria	any	Optional. Comparer for equality testing, e.g. Comparer.OrdinalIgnoreCase.
Returns	Returns true if value is found in list.	
Example	List.Contains({"East","West","Central"}, "West")	
Result	true	
NOTE	The most common use: List.Contains(validList, [ColumnValue]) inside Table.SelectRows to keep only rows whose column value appears in a known-good list. Pass Comparer.OrdinalIgnoreCase for case-insensitive text matching.	

List.ContainsAll	Returns true only if ALL values in the second list are present in the first list.
List.ContainsAll(list, values, optional equationCriteria)	List.ContainsAll({"a", "b", "c", "d"}, {"a", "c"}) → true
List.ContainsAny	Returns true if AT LEAST ONE value in the second list is present in the first list.
List.ContainsAny(list, values, optional equationCriteria)	List.ContainsAny({"a", "b", "c"}, {"x", "b", "z"}) → true

Searching: List.PositionOf and List.PositionOfAny

List.PositionOf

Syntax	List.PositionOf(list as list, value as any, optional occurrence as nullable number, optional equationCriteria as any) as any	
Parameter	Type	Description
list	list	The list to search.
value	any	The value to find.
occurrence	nullable number	Optional. Occurrence.First (default) or Occurrence.Last.
equationCriteria	any	Optional. Comparer for equality.

Returns	Returns the zero-based index of the first (or last) occurrence of value. Returns -1 if not found.
Example	List.PositionOf({"East", "West", "Central", "West"}, "West")
Result	1
NOTE Use List.PositionOf to find the index of a value for subsequent range operations. Returns -1 (not an error) when the value is not found — always check for -1 before using the result as an index.	

List.PositionOfAny	Returns the position of the first occurrence of any value from the second list.
List.PositionOfAny(list, values, optional occurrence, optional equationCriteria)	List.PositionOfAny({"a", "b", "c", "d"}, {"c", "b"}) → 1 (position of first match)

Statistical Inspection

These functions return a single value summarising the list. They are faster than building a one-row table and aggregating — use them for scalar intermediate values in complex queries:

List.Min	Returns the minimum value. Works on numbers, dates, text. Returns default if list is empty.
List.Min(list, optional default, optional comparisonCriteria)	List.Min({5, 2, 8, 1, 9}) → 1
List.Max	Returns the maximum value. Accepts an optional custom comparer for non-standard comparisons.
List.Max(list, optional default, optional comparisonCriteria)	List.Max({5, 2, 8, 1, 9}) → 9
List.Median	Returns the median (middle value). For even-count lists, returns the lower of the two middle values.
List.Median(list, optional comparisonCriteria)	List.Median({1, 2, 3, 4, 5}) → 3
List.Mode	Returns a list of the most frequently occurring values (there may be multiple modes).
List.Modes(list, optional equationCriteria)	List.Modes({1, 2, 2, 3, 3, 3, 4}) → {3} (most frequent value as a list)
List.Percentile	Returns the value at the specified percentile (0.0-1.0). Pass a list of percentiles for multiple results.
List.Percentile(list, percentiles, optional options)	List.Percentile({1, 2, 3, 4, 5, 6, 7, 8, 9, 10}, 0.9) → 9

LESSON 4

Transforming Lists

After completing this lesson, you will be able to:

- Apply a function to every element with List.Transform.
- Filter list elements by condition using List.Select.
- Sort a list with List.Sort and a custom comparer.
- Remove duplicates with List.Distinct.
- Skip, trim, and slice lists using List.Skip, List.RemoveFirstN, List.LastN.
- Remove null values and specific items from a list.
- Pair elements across two lists using List.Zip.

List.Transform — Apply a Function to Every Element

List.Transform

Syntax	List.Transform(list as list, transform as function) as list	
Parameter	Type	Description
list	list	The source list.
transform	function	A function applied to each element. Use each for the shorthand form.
Returns	Returns a new list where every element is the result of applying transform.	
Example	List.Transform({1, 2, 3, 4}, each _ * _)	
Result	{1, 4, 9, 16}	
NOTE	List.Transform is the list equivalent of Table.TransformColumns. The each keyword's _ refers to the current list element (not a row record). This is one of the most frequently used list functions in advanced M patterns.	

List.Transform examples:

```
// Convert a list of numbers to text
NumberStrings = List.Transform({1,2,3,4,5}, Text.From)
// = {"1", "2", "3", "4", "5"}
```

```
// Apply a complex expression
Doubled = List.Transform({10, 20, 30}, each _ * 2)
// = {20, 40, 60}

// Transform a list of dates to month names
DateList = {#date(2024,1,1), #date(2024,2,1), #date(2024,3,1)}
MonthNames = List.Transform(DateList, each Date.MonthName(_, "en-US"))
// = {"January", "February", "March"}

// Clean a list of strings
RawNames = {" alice ", "BOB ", " Carol"}
CleanNames = List.Transform(RawNames, each Text.Proper(Text.Trim(_)))
// = {"Alice", "Bob", "Carol"}
```

List.Select — Filter Elements by Condition

List.Select

Syntax	List.Select(list as list, condition as function) as list	
Parameter	Type	Description
list	list	The source list.
condition	function	A function returning logical. each _ > 5 keeps elements > 5.
Returns	Returns a new list containing only the elements for which condition returns true.	
Example	List.Select({1, 5, 10, 3, 8, 2}, each _ > 4)	
Result	{5, 10, 8}	

NOTE

List.Select is the list equivalent of Table.SelectRows. Common uses: filter a list of column names, filter a list of dates to weekdays only, filter a list of file paths by extension.

```
// Keep only non-null values
NoNulls = List.Select(SomeList, each _ <> null)
// (equivalent to List.RemoveNulls)

// Keep only weekday dates from a date list
Weekdays = List.Select(DateList,
each Date.DayOfWeek(_, Day.Monday) < 5)

// Keep only column names that start with 'Sales'
SaleCols = List.Select(Table.ColumnNames(Source),
each Text.StartsWith(_, "Sales"))

// Keep only values within 2 standard deviations of the mean
Mean = List.Average(Data)
```

```

StDev = List.StandardDeviation(Data)
Clean = List.Select(Data,
each Number.Abs(_ - Mean) <= 2 * StDev)

```

List.Sort — Ordering Lists

List.Sort

Syntax `List.Sort(list as list, optional comparisonCriteria as any) as list`

Parameter	Type	Description
<code>list</code>	<i>list</i>	The source list.
<code>comparisonCriteria</code>	<i>any</i>	Optional. Order.Ascending (default), Order.Descending, or a custom comparison function.

Returns Returns a new sorted list. Does not modify the original.

Example `List.Sort({5,1,8,3,9,2})`

Result `{1, 2, 3, 5, 8, 9}`

NOTE For descending sort: `List.Sort(list, Order.Descending)`. For custom sort: pass a function (x, y) => if x > y then 1 else if x < y then -1 else 0. This is used to sort lists of records or complex objects.

List.Distinct	Removes duplicate values. Preserves the first occurrence. Optional comparer for case-insensitive dedup.
<code>List.Distinct(list, optional equationCriteria)</code>	<code>List.Distinct({"a","b","a","c","b"}) → {"a", "b", "c"}</code>
List.Reverse	Returns the list in reverse order.
<code>List.Reverse(list)</code>	<code>List.Reverse({1,2,3,4,5}) → {5, 4, 3, 2, 1}</code>
List.Skip	Skips the first n elements (or skips while a condition is true) and returns the rest.
<code>List.Skip(list, countOrCondition)</code>	<code>List.Skip({10,20,30,40,50}, 2) → {30, 40, 50}</code>
List.RemoveFirstN	Removes the first n elements. Equivalent to List.Skip.
<code>List.RemoveFirstN(list, countOrCondition)</code>	<code>List.RemoveFirstN({1,2,3,4,5}, 2) → {3, 4, 5}</code>
List.RemoveLastN	Removes the last n elements from the list.
<code>List.RemoveLastN(list, countOrCondition)</code>	<code>List.RemoveLastN({1,2,3,4,5}, 2) → {1, 2, 3}</code>
List.RemoveNulls	Removes all null values. Equivalent to <code>List.Select(list, each _ <> null)</code> .

List.RemoveNulls(list)	List.RemoveNulls({1, null, 3, null, 5}) → {1, 3, 5}
List.RemoveItems	Removes all elements that appear in the second list.
List.RemoveItems(list, itemsToRemove)	List.RemoveItems({"a", "b", "c", "d"}, {"b", "d"}) → {"a", "c"}
List.RemoveMatchingItems	Same as List.RemoveItems but with a custom comparer for case-insensitive or culture-aware matching.
List.RemoveMatchingItems(list, itemsToRemove, optional equationCriteria)	List.RemoveMatchingItems({"A", "b", "C"}, {"a", "c"}, Comparer.OrdinalIgnoreCase) → {"b"}
List.Repeat	Repeats the list count times, concatenating the repetitions.
List.Repeat(list, count)	List.Repeat({"A", "B"}, 3) → {"A", "B", "A", "B", "A", "B"}
List.ReplaceValue	Replaces occurrences of oldValue with newValue. Use Replacer.ReplaceValue for exact matching.
List.ReplaceValue(list, oldValue, newValue, replacer)	List.ReplaceValue({1,2,null,4}, null, 0, Replacer.ReplaceValue) → {1, 2, 0, 4}

List.Zip — Pairing Elements Across Lists

List.Zip		
Syntax	List.Zip(lists as list) as list	
Parameter	Type	Description
lists	list	A list of lists to zip together. All sublists should have the same length.
Returns	Returns a list of lists where each sublist contains the n-th element from each input list.	
Example	List.Zip({{1,2,3}, {"a","b","c"}})	
Result	{ {1,"a"}, {2,"b"}, {3,"c"} }	
NOTE List.Zip is used to pair column names with values, pair keys with values for record construction, or pair parallel lists before further processing. If the lists have different lengths, the shorter one determines the output length.		

```
// Common pattern: pair column names with values to build a record
ColNames = {"Name", "Score", "Passed"}
Values = {"Alice", 92, true}
Zipped = List.Zip({ColNames, Values})
// = {{"Name","Alice"}, {"Score",92}, {"Passed",true}}
```

```
// Convert to a record:  
AsRecord = Record.FromList(Values, ColNames)  
// = [Name="Alice", Score=92, Passed=true]
```

LESSON 5

Aggregating Lists

After completing this lesson, you will be able to:

- Compute sum, average, min, and max of a numeric list.
- Calculate standard deviation, variance, and covariance.
- Retrieve the median, mode, and percentile of a list.
- Build custom aggregations with List.Accumulate.

Standard Aggregation Functions

All aggregation functions accept a list and return a scalar value. They work directly on table columns via the table[Column] syntax, making them ideal for quick scalar calculations inside other expressions:

List.Sum

Syntax	List.Sum(list as list, optional precision as nullable number) as any	
Parameter	Type	Description
list	list	A list of numeric or duration values.
precision	nullable number	Optional. Precision.Double (default) or Precision.Decimal.
Returns	Returns the sum of all values in the list. Nulls are ignored.	
Example	List.Sum({10, 20, 30, null, 40})	
Result	100	
NOTE	Nulls are silently skipped. List.Sum({}) returns 0. Also works on duration lists: List.Sum({#duration(1,0,0,0), #duration(2,0,0,0)}) = #duration(3,0,0,0). Use Precision.Decimal for financial summations to avoid floating-point drift.	

List.Average

Syntax	List.Average(list as list, optional precision as nullable number) as any	
Parameter	Type	Description
list	list	A list of numeric values.

precision	nullable number	Optional. Precision.Double or Precision.Decimal.
Returns	Returns the arithmetic mean of all values. Nulls are ignored.	
Example	List.Average({10, 20, 30, 40})	
Result	25	
NOTE Nulls are excluded from both the sum and the count. List.Average({}) returns null — check for empty list if this could occur.		

List.StandardDeviation	Returns the population standard deviation. Returns null if fewer than 2 non-null values.
<code>List.StandardDeviation(list)</code>	<code>List.StandardDeviation({2, 4, 4, 4, 5, 5, 7, 9})</code> → 2
List.Variance	Returns the population variance (standard deviation squared).
<code>List.Variance(list)</code>	<code>List.Variance({2, 4, 4, 4, 5, 5, 7, 9})</code> → 4
List.Covariance	Returns the covariance of two equal-length lists. Used for correlation analysis.
<code>List.Covariance(list1, list2)</code>	<code>List.Covariance({2, 4, 6}, {3, 5, 7})</code> → 2.666...
List.Correlation	Returns the Pearson correlation coefficient (-1 to 1) between two equal-length numeric lists.
<code>List.Correlation(list1, list2)</code>	<code>List.Correlation({2, 4, 6}, {3, 5, 7})</code> → 1.0 (perfect correlation)
List.Product	Returns the product (multiplication) of all values in the list. Nulls are treated as 1.
<code>List.Product(list)</code>	<code>List.Product({2, 3, 4})</code> → 24

List.Accumulate — Custom Aggregations and State Machines

`List.Accumulate` is the most powerful aggregation function in M. It processes a list element-by-element, maintaining a running state — equivalent to a reduce/fold operation in functional programming. It is used for running totals, string concatenation, custom aggregations, and any computation that depends on all previous elements:

List.Accumulate

Syntax	<code>List.Accumulate(list as list, seed as any, accumulator as function)</code> as any	
Parameter	Type	Description
<code>list</code>	<i>list</i>	The list of values to process.
<code>seed</code>	<i>any</i>	The initial state value before processing begins.

accumulator	function	(state, current) => newState — Called for each element. state is the running accumulation, current is the current list element.
Returns	Returns the final state after applying accumulator to every element.	
Example	List.Accumulate({1,2,3,4,5}, 0, (state,current) => state + current)	
Result	15 (same as List.Sum)	
NOTE List.Accumulate's power comes from arbitrary state: the state can be a number, a text string, a list, or a record — enabling complex multi-variable accumulations.		

List.Accumulate patterns:

```
// Running concatenation (join all strings)
Words = {"Power", "Query", "M", "Language"}
Sentence = List.Accumulate(Words, "",
(state, current) => if state = "" then current
else state & " " & current)
// = "Power Query M Language"

// Running maximum (equivalent to List.Max, but illustrates the pattern)
Numbers = {5, 2, 8, 1, 9, 3}
RunMax = List.Accumulate(Numbers, Numbers{0},
(state, current) => if current > state then current else state)
// = 9

// Count elements satisfying a condition (without List.Select + List.Count)
Values = {1, 5, 2, 8, 3, 7, 4}
CountAbove5 = List.Accumulate(Values, 0,
(state, current) => if current > 5 then state + 1 else state)
// = 2 (values 8 and 7 are above 5)

// Build a running total list (cumulative sum)
Sales = {100, 150, 200, 175, 225}
CumSum = List.Skip( // skip the seed
List.Accumulate(Sales, {0},
(state, current) => state & {List.Last(state) + current}
),
1 // remove the initial 0 seed
)
// = {100, 250, 450, 625, 850}
```

WARNING List.Accumulate with a list as the seed/state (building a running total list) can be slow for large datasets because each step creates a new list. For production running totals, use a DAX measure with CALCULATE and ALLSELECTED instead — it is far more performant and responds to slicer context. Reserve List.Accumulate for M-side computations where DAX is not available.

LESSON 6

Set Operations and Combining Lists

After completing this lesson, you will be able to:

- Combine lists without duplicates using List.Union.
- Find common elements with List.Intersect.
- Find elements present in one list but not another with List.Difference.
- Concatenate lists with List.Combine and the & operator.
- Convert between lists and tables using List.ToTable and Table.ToList.

Set Operation Functions

Set operations treat lists as mathematical sets — unordered collections of distinct values. They are powerful tools for data validation, comparing column sets, and building dynamic filter lists:

List.Union

Syntax	List.Union(lists as list, optional equationCriteria as any) as list	
Parameter	Type	Description
lists	list	A list of lists to union together.
equationCriteria	any	Optional. Custom equality comparer.
Returns	Returns a list of all distinct values across all input lists (no duplicates).	
Example	List.Union({{1,2,3}, {2,3,4,5}, {5,6}})	
Result	{1, 2, 3, 4, 5, 6}	
NOTE	List.Union deduplicates across all lists. It is the correct way to merge multiple lists from different sources and ensure no duplicates. Note: the input is a LIST of lists, not individual lists as arguments.	

List.Intersect

Syntax	List.Intersect(lists as list, optional equationCriteria as any) as list	
Parameter	Type	Description

lists	list	A list containing exactly two (or more) lists to intersect.
equationCriteria	any	Optional. Custom equality comparer.
Returns	Returns a list of values present in ALL input lists.	
Example	List.Intersect({{1, 2, 3, 4}, {2, 4, 6}})	
Result	{2, 4}	
NOTE Use to find products in multiple category lists, find columns present in all source files, or find customers who appear in multiple cohorts.		

List.Difference

Syntax	List.Difference(list1 as list, list2 as list, optional equationCriteria as any) as list	
Parameter	Type	Description
list1	list	The source list.
list2	list	Values to remove from list1.
equationCriteria	any	Optional. Custom equality comparer.
Returns	Returns a list of values in list1 that are NOT in list2.	
Example	List.Difference({1,2,3,4,5}, {2,4})	
Result	{1, 3, 5}	
NOTE	This is a set subtraction (list1 minus list2). A key use case: find column names that exist in source but are NOT in the expected schema — a data quality check. List.Difference(Table.ColumnNames(Source), ExpectedColumns) shows unexpected columns.	

Combining and Converting Lists

List.Combine

Syntax	<code>List.Combine(lists as list) as list</code>	
Parameter	Type	Description
<code>lists</code>	<code>list</code>	A list of lists to concatenate in order.
Returns	Returns a single list by concatenating all input lists in order (preserving duplicates).	
Example	<code>List.Combine({{1,2,3}, {4,5}, {6,7,8}})</code>	
Result	<code>{1, 2, 3, 4, 5, 6, 7, 8}</code>	

NOTE `List.Combine` preserves all elements including duplicates — unlike `List.Union` which deduplicates. Use `List.Combine` when you need to merge lists and preserve order/duplicates. The `&` operator is syntactic sugar for `List.Combine` of two lists.

List.ToTable	Converts a list to a single-column table. Useful for turning a list into a filterable/joinable table.
<code>List.ToTable(list, optional splitFunc, optional colNames)</code>	<code>List.ToTable({"East", "West", "Central"}) → table with one column "Column1" and 3 rows</code>
Table.ToList	Converts each row of a table to a single value by combining columns. The combiner determines how columns are merged.
<code>Table.ToList(table, optional combiner)</code>	<code>Table.ToList(#table({"A", "B"}, {{1, 2}, {3, 4}}), Combiner.CombineTextByDelimiter(", ")) → {"1,2", "3,4"}</code>
Record.FromList	Creates a record from a values list and a field names list. Useful for dynamic record construction.
<code>Record.FromList(list, fields)</code>	<code>Record.FromList({1, "Alice", true}, {"ID", "Name", "Active"}) → [ID=1, Name="Alice", Active=true]</code>
Record.FieldValues	Extracts all field values from a record as a list. Pair with <code>Record.FieldNames</code> for full record decomposition.
<code>Record.FieldValues(record)</code>	<code>Record.FieldValues([A=1, B=2, C=3]) → {1, 2, 3}</code>
Record.FieldNames	Returns a list of all field names in the record. Use with <code>Table.ColumnNames</code> for schema inspection.
<code>Record.FieldNames(record)</code>	<code>Record.FieldNames([A=1, B=2, C=3]) → {"A", "B", "C"}</code>

LESSON 7

Advanced List Patterns for PL-300

After completing this lesson, you will be able to:

- Use List.Accumulate to build running totals and state machines.
- Use List.Generate to create parameterised sequences and paginate API requests.
- Dynamically select and reorder table columns from a configuration list.
- Implement a lookup without a table join using list functions.
- Filter a table on multi-value parameters using List.Contains.
- Build a cross-join table from two lists.

Pattern 1: Dynamic Column Selection

Hard-coding column names is fragile — source schemas change. Using a list to drive Table.SelectColumns makes queries schema-resilient:

■ Dynamic Column Selection with Schema Validation

```
let
Source = /* raw data */,
// Define the columns we WANT (in desired output order)
WantedColumns = {"OrderID", "OrderDate", "CustomerName",
"Region", "Sales", "Profit"},

// Find which wanted columns actually exist in the source
AvailableCols = Table.ColumnNames(Source),
PresentCols = List.Intersect({WantedColumns, AvailableCols}),

// Find wanted columns that are MISSING (data quality alert)
MissingCols = List.Difference(WantedColumns, AvailableCols),

// Select only the present wanted columns, in WantedColumns order
// List.Sort to respect WantedColumns order:
OrderedPresent = List.Select(WantedColumns, each List.Contains(AvailableCols,
_)),
Result = Table.SelectColumns(Source, OrderedPresent,
MissingField.Ignore)
in
Result // Change to MissingCols to see what is absent
```

This pattern prevents failures when source columns are renamed or removed. MissingCols reveals schema drift before it causes downstream errors. List.Select on WantedColumns (not AvailableCols) preserves the desired output column order.

Pattern 2: Lookup Without a Table Join

Sometimes you need to look up a value from a small reference set without creating a full merge/join operation. List.PositionOf combined with a parallel value list delivers a fast inline lookup:

■ Inline Lookup Using Parallel Lists

```
let
Source = /* table with RegionCode column */,

// Define parallel key/value lookup lists
RegionCodes = {"E", "W", "C", "S", "NE", "NW"},
RegionNames = {"East", "West", "Central", "South",
"North East", "North West"},

// Lookup function: find code in key list, return name at same position
LookupRegion = (code as text) as text =>
let
Pos = List.PositionOf(RegionCodes, code)
in
if Pos = -1 then "Unknown"
else RegionNames{Pos},

// Apply to table
WithRegionName = Table.AddColumn(Source, "RegionName",
each LookupRegion([RegionCode]), type text)
in
WithRegionName
```

This is faster than a full Table.Join for small reference tables (< 100 rows). The key and value lists must be parallel (same index = same pair). For larger lookups, use Table.Join or a proper dimension table instead.

Pattern 3: Multi-Value Parameter Filtering

A common Power BI development requirement is filtering a table on a parameter that can hold multiple values (e.g. 'show data for these regions'). List.Contains drives this pattern:

■ Multi-Value Parameter Filter

```
let
Source = /* FactSales table */,

// Parameter: which regions to include
// In a real model, this could be loaded from a Parameters table
SelectedRegions = {"East", "West"},
```

```
// Filter: keep rows where Region appears in the selected list
Filtered = Table.SelectRows(Source,
each List.Contains(SelectedRegions, [Region])),

// OR: exclude certain regions
Excluded = Table.SelectRows(Source,
each not List.Contains({"Central"}, [Region]))
in
Filtered
```

Changing SelectedRegions to {} (empty list) returns zero rows — useful for testing. Change to Table.ColumnNames(DimRegion) for a dynamic list driven by a dimension table. This is the foundational pattern for Query Parameters in Power BI report filters.

Pattern 4: Building a Cross-Join Table from Two Lists

A cross-join produces every combination of two sets. This is useful for building scenario grids, what-if tables, or test data:

■ Cross-Join from Two Lists

```
let
Products = {"Laptop", "Tablet", "Phone"},
Regions = {"East", "West", "Central"},

// Generate all product-region combinations
CrossJoin = List.Combine(
List.Transform(Products, (product) =>
List.Transform(Regions, (region) =>
[Product = product, Region = region]
)
),

// Convert list of records to a table
AsTable = Table.FromRecords(CrossJoin)
in
AsTable
// = 9 rows: every Product x Region combination
```

This produces a 3x3 = 9 row table with Product and Region columns. Scale to any list sizes. The outer List.Transform iterates over Products, the inner iterates over Regions, creating a list of records for each product-region pair. List.Combine flattens the nested result.

Pattern 5: Distinct Values from Multiple Columns

Getting distinct values across multiple columns is a common requirement for building dimension tables from a fact table:

■ All Distinct Values Across Multiple Columns

```
// Get every unique Region and SubRegion combination as a single list
// (useful for building a Region dimension from a denormalised fact)
let
    Source = FactSales,

    // Get distinct Region values
    Regions = List.Distinct(Source[Region]),

    // Get distinct values from MULTIPLE columns combined
    AllLocations = List.Distinct(
        List.Combine({Source[Region], Source[City]}
    ),

    // Or build a proper dimension by selecting distinct column combos
    RegionDim = Table.Distinct(
        Table.SelectColumns(Source, {"Region", "SubRegion"})
    )
in
    RegionDim
```

Table.Distinct deduplicates whole rows — the correct approach for dimension extraction. List.Distinct operates on a single column. List.Combine then List.Distinct aggregates unique values across multiple columns into a flat list.

DEMONSTRATION 1**Dynamic Column Selection and Reordering**

In this demonstration the instructor builds a schema-resilient query that selects and reorders columns dynamically from a configuration list, and reports any missing columns as a data quality alert.

Scenario: The Contoso ERP export adds and removes columns unpredictably. Your query must always produce a consistent output schema regardless of what the source delivers.

Demonstration Steps**Step 1: Load the source file**

Open Power BI Desktop. Load Contoso_Products.csv. Note the column order and names — they may differ from what the model expects.

Step 2: Inspect Table.ColumnNames

In the Advanced Editor, add: `SourceCols = Table.ColumnNames(Source)`. Temporarily set 'in SourceCols' and observe the list of available columns.

Step 3: Define the WantedColumns list

Add a step defining a list of required columns in the desired output order: `WantedColumns = {"ProductID", "ProductName", "Category", "SubCategory", "UnitPrice", "Supplier"}`.

Step 4: Compute intersection and difference

Add: `PresentCols = List.Intersect({WantedColumns, SourceCols})`. Add: `MissingCols = List.Difference(WantedColumns, SourceCols)`. Change 'in' to `MissingCols` to see which columns are absent.

Step 5: Apply ordered selection

Add: `OrderedCols = List.Select(WantedColumns, each List.Contains(SourceCols, _))`. Then: `Result = Table.SelectColumns(Source, OrderedCols, MissingField.Ignore)`.

Step 6: Test resilience

Rename a column in the source file to simulate schema drift. Re-run the query. The `MissingCols` list now shows the missing column name. The `Result` table excludes it gracefully without erroring.

Step 7: Add a warning step

Add a conditional step that raises a meaningful error message if `MissingCols` is not empty: if `List.IsEmpty(MissingCols)` then `Result` else `error "Missing columns: " & Text.Combine(MissingCols, ", ")`.

RESULT The query dynamically selects and orders columns from the source, is resilient to schema changes, and optionally raises a clear data quality error when required columns are absent. No hard-coded column indices used anywhere.

DEMONSTRATION 2**Paginated API Ingestion with List.Generate**

In this demonstration the instructor builds a paginated data retrieval pattern using List.Generate to repeatedly call a data source until all pages are exhausted — the canonical technique for REST API pagination in Power Query.

Scenario: A product API returns 100 records per page. The total number of pages is unknown in advance. We must fetch page 1, then 2, 3... until an empty response is returned.

Demonstration Steps**Step 1: Create a GetPage custom function**

Create a blank query named GetPage. Define it as: (pageNum as number) => Web.Contents("https://api.example.com/products?page=" & Text.From(pageNum)). For this demo, simulate with local JSON files.

Step 2: Build the List.Generate loop

Create a new blank query. Add the List.Generate call with initial = () => [Page=1, Data=GetPage(1)], condition = each Table.RowCount([Data]) > 0, next = each [Page=[Page]+1, Data=GetPage([Page]+1)], selector = each [Data].

Step 3: Handle the termination condition

Wrap GetPage in a try...otherwise to return an empty table when the API returns a 404 or empty body. This prevents errors from crashing the List.Generate loop.

Step 4: Combine all pages

Add: Combined = Table.Combine(AllPages). Observe that all page data is combined into a single table.

Step 5: Inspect the query folding behaviour

Open Performance Analyzer in Power BI Desktop and refresh. Note that List.Generate prevents query folding — all data is fetched by the M engine, not pushed to the source. This is expected for APIs.

Step 6: Add page count metadata

Add: PageCount = List.Count(AllPages). Use this in a step comment or a separate query for monitoring purposes.

RESULT

A single table contains all records from all pages of the API. The query handles unknown page counts, handles empty/error pages gracefully, and combines all results. This is the canonical Power Query API pagination pattern.

LAB 4

List Functions

In this lab you apply the full range of List.* functions — from basic creation and inspection through transformation, set operations, and the advanced List.Accumulate and List.Generate patterns. Work with the FactSales, DimProduct, and GlobalSales queries from previous labs.

Lab Scenario

Contoso Analytics needs several enhancements to their Power BI model. You must: validate the incoming product column schema dynamically, build a running cumulative sales column, create a product-region planning cross-join table, and generate a controlled sequence of forecast dates using List.Generate.

NOTE All exercises use M queries only — no ribbon clicks. Write all code in the Advanced Editor. Label every step clearly.

Exercise 1: List Creation, Inspection, and Indexing (Estimated time: 20 minutes)

In this exercise you practice creating lists and navigating them.

1. Create a **blank query named ListBasics**. In the Advanced Editor write a let...in with the following named steps:
2. **MonthNumbers** = List.Numbers(1, 12, 1) — a list of months 1 through 12.
3. **MonthNames** = List.Transform(MonthNumbers, each Date.MonthName(#date(2024, _, 1), "en-US")) — convert to month name strings.
4. **Q2Months** = List.Range(MonthNames, 3, 3) — extract April, May, June by position.
5. **FiscalMonths** = List.Combine({List.Skip(MonthNames, 9), List.FirstN(MonthNames, 9)}) — reorder so October is first (fiscal year starting October).
6. **HasJuly** = List.Contains(MonthNames, "July") — verify July exists.
7. **JulyPosition** = List.PositionOf(MonthNames, "July") — find its zero-based index.
8. **LastMonth** = List.Last(MonthNames) — safely get the last month.
9. Change the **in** clause to each step in turn and verify the results. Final output: **in FiscalMonths**.

RESULT ListBasics returns a 12-element list starting with 'October', 'November', ..., 'September'.
T HasJuly = true. JulyPosition = 6. LastMonth = 'December'.

Exercise 2: List Transformation and Filtering (Estimated time: 20 minutes)

In this exercise you apply `List.Transform` and `List.Select` to real data.

1. Select the **FactSales** query. Add a blank query named **SalesAnalysis**.
2. Add: **SalesValues** = `FactSales[Sales]` — extract the Sales column as a list.
3. Add: **RoundedSales** = `List.Transform(SalesValues, each Number.Round(_, 0))` — round every value to nearest integer.
4. Add: **HighSales** = `List.Select(SalesValues, each _ > 1000)` — keep only values over 1000.
5. Add: **SortedTop10** = `List.FirstN(List.Sort(SalesValues, Order.Descending), 10)` — top 10 sales values.
6. Add: **NoNulls** = `List.RemoveNulls(FactSales[Profit])` — remove null profits.
7. Add: **UniqueRegions** = `List.Distinct(FactSales[Region])` — get unique region names.
8. Add: **SortedRegions** = `List.Sort(UniqueRegions)` — sort alphabetically.
9. Add: **RegionCount** = `List.Count(UniqueRegions)` — how many distinct regions?
10. Change **in** to **SortedTop10**. Verify the 10 highest sales values appear in descending order.

RESULT SortedTop10 shows the 10 highest sales values in descending order. SortedRegions shows the unique regions alphabetically. RegionCount matches the number of distinct values in the Region column.

Exercise 3: List Aggregation and Set Operations (Estimated time: 20 minutes)

In this exercise you aggregate lists and apply set operations.

1. In **SalesAnalysis**, add: **TotalSales** = `List.Sum(FactSales[Sales])`.
2. Add: **AvgSales** = `Number.Round(List.Average(FactSales[Sales]), 2)`.
3. Add: **StDevSales** = `Number.Round(List.StandardDeviation(FactSales[Sales]), 2)`.
4. Add: **Pct90** = `List.Percentile(FactSales[Sales], 0.9)` — the 90th percentile sales value.
5. Add: **FSSales** = `List.Select(FactSales[Sales], each _ > Pct90)` — values above 90th percentile.
6. Add: **FactRegions** = `List.Distinct(FactSales[Region])`.
7. Add: **ExpectedRegions** = {"East", "West", "Central", "South"}.
8. Add: **UnexpectedRegions** = `List.Difference(FactRegions, ExpectedRegions)` — any surprise values?
9. Add: **MissingRegions** = `List.Difference(ExpectedRegions, FactRegions)` — any expected regions absent?
10. Add: **AllRegions** = `List.Union({FactRegions, ExpectedRegions})` — combined unique set.

11. Set in UnexpectedRegions. If empty, data is clean. If not empty, identify the unexpected values.

RESULT TotalSales, AvgSales, StDevSales, and Pct90 are computed correctly. UnexpectedRegions is either empty (clean data) or shows unexpected values. MissingRegions shows any expected regions with no sales. AllRegions is the deduplicated union.

Exercise 4: Advanced Patterns — List.Accumulate and List.Generate (Estimated time: 25 minutes)

In this exercise you implement the two most powerful list patterns.

1. Create a blank query named **CumulativeSales**.
2. Add: **MonthlySales = {120000, 145000, 98000, 167000, 134000, 189000, 201000, 155000, 143000, 176000, 210000, 225000}** — 12 monthly values.
3. Use **List.Accumulate** to build a running cumulative total list. Seed = {0}. Accumulator = (state, current) => state & {List.Last(state) + current}. Then use List.Skip to remove the leading 0 seed.
4. Add: **CumPercent = List.Transform(CumulativeSales, each Number.Round(_ / List.Last(CumulativeSales) * 100, 1))** — cumulative % of annual total.
5. Convert the result to a table: **CumTable = Table.FromColumns({List.Numbers(1,12,1), MonthlySales, CumulativeSalesResult, CumPercent}, {"Month", "MonthlySales", "CumulativeTotal", "CumulativePct"}).**
6. Create a blank query named **ForecastDates**.
7. Use **List.Generate** to build the next 12 month-start dates from today: initial = () => Date.StartOfMonth(Date.AddMonths(DateTime.Date(DateTime.LocalNow()), 1)), condition = each _ <= Date.AddMonths(DateTime.Date(DateTime.LocalNow()), 12), next = each Date.AddMonths(_, 1).
8. Convert ForecastDates to a table and add a MonthName column using Date.MonthName.
9. Create a blank query named **ProductRegionGrid**. Implement the cross-join pattern from Lesson 7 Pattern 4 using Products = List.Distinct(FactSales[Category]) and Regions = List.Distinct(FactSales[Region]).
10. Load all three queries (CumulativeSales, ForecastDates, ProductRegionGrid). Verify: CumTable has 12 rows ending at 100%; ForecastDates has 12 rows; ProductRegionGrid has Category x Region rows (e.g. 4 categories x 4 regions = 16 rows).

RESULT CumulativeSales shows monthly and cumulative totals, with CumulativePct reaching 100% in month 12. ForecastDates generates the next 12 calendar month starts from today dynamically. ProductRegionGrid contains every category-region combination as a planning matrix. All three tables load into the Power BI model.

Module Review and Takeaways

Review Questions

What is the zero-based index of the third element in a list?

Answer: Index 2. Lists in M are zero-indexed: first element = {0}, second = {1}, third = {2}.

What is the difference between List.Combine and List.Union?

Answer: List.Combine concatenates lists in order while preserving all duplicates. List.Union combines lists and removes all duplicate values, returning only distinct elements. Use List.Combine when you need to append lists; use List.Union when you need a deduplicated merged set.

What is the difference between List.Intersect and List.Difference?

Answer: List.Intersect returns elements present in ALL input lists (the common subset). List.Difference(A, B) returns elements in A that are NOT in B (A minus B). They are complementary: $A = \text{Intersect} + \text{Difference}(A, B)$.

How does List.Generate work and what are its four parameters?

Answer: List.Generate iterates like a while loop. initial() creates the starting state; condition(state) returns true while iteration should continue; next(state) produces the next state; selector(state) optionally extracts the output value from each state (if omitted, the state is the output). It stops when condition returns false.

What does List.Accumulate return and what are its three parameters?

Answer: It returns a single final value (the accumulated result). Parameters: list (to iterate over), seed (initial state), accumulator (state, current) => newState. Each call to accumulator receives the running state and the current list element.

How would you filter a table to keep only rows where a column value appears in a given list?

Answer: Table.SelectRows(table, each List.Contains(allowedList, [ColumnName])). This is the multi-value parameter filter pattern.

What function extracts a table column as a list?

Answer: The table[ColumnName] syntax: FactSales[Region] returns a list of all Region values. Wrap with List.Distinct() for unique values only.

Why should you be cautious when using List.Accumulate to build a running total list?

Answer: Each iteration creates a new, slightly larger list. For large datasets (thousands of rows), this causes quadratic memory growth and very slow performance. Use DAX measures with CALCULATE for running totals on large datasets. List.Accumulate is best for small lists or non-table M computations.

Best Practices

- Always pass a default value to `List.First` and `List.Last` when the list might be empty. `List.First({}, null)` is safe; `List.First({})` throws an error if the list is empty.
- Use `table[ColumnName]` to extract a column as a list rather than `Table.ToList` — it is more readable and does not require a combiner function.
- Prefer `List.Distinct` over a `Table.Distinct` one-column approach when you just need unique values from a column. It is more concise and equally performant.
- Use `List.Contains` inside `Table.SelectRows` for multi-value filtering. This is cleaner and more maintainable than long chains of 'or' conditions.
- For set validation (checking source columns against an expected schema), always use `List.Difference` in both directions: `missing = Difference(expected, actual)`, `unexpected = Difference(actual, expected)`.
- Avoid large `List.Accumulate` loops (>1000 elements) that build lists as state. The quadratic memory pattern can cause OOM errors or very slow refreshes. Use `Table.AddIndexColumn` and a DAX measure instead.
- When using `List.Generate` for pagination, always include a safety limit in the condition (e.g. `and [Page] < 1000`) to prevent infinite loops if the API termination signal is missing.
- Use `List.Buffer` to cache a list that is referenced multiple times in the same query. This prevents the list from being re-evaluated on each access.
- For ordered output, `List.Sort` returns a new list — the original is unchanged. Remember to assign the result: `SortedList = List.Sort(OriginalList)`.
- Use `List.IsEmpty` before `List.First` when there is any chance the list might be empty. Pattern: `if List.IsEmpty(myList) then null else List.First(myList)`.

Common Issues and Solutions

List index out of range error

Cause: Accessing `list{n}` where `n >= List.Count(list)`. Most commonly when the list is empty or shorter than expected.

Solution: Use `List.First(list, null)` instead of `list{0}`. For arbitrary index `n`: `if List.Count(list) > n then list{n} else null`.

List.Generate runs forever / never terminates

Cause: The condition function never returns false — either the state never reaches the termination criterion, or the API always returns a non-empty response.

Solution: Add a maximum iteration guard to the condition: `each [Data] <> null and [Page] < 500`. This caps the loop at 500 iterations as a safety net.

List.Accumulate is very slow on large data

Cause: Building a list as the accumulator state causes $O(n^2)$ memory allocations as `n` grows.

Solution: For running totals on large fact tables, use a DAX measure instead: Running Total = CALCULATE(SUM(Table[Value]), FILTER(ALLSELECTED(DimDate), DimDate[Date] <= MAX(DimDate[Date]))).

List.Union or List.Intersect returns unexpected results

Cause: The input to List.Union/Intersect must be a list of lists, not individual list arguments: List.Union({list1, list2}) — note the outer braces wrapping both lists.

Solution: Always wrap the input lists in an outer list literal: List.Union({ListA, ListB}). A common mistake is writing List.Union(ListA, ListB) which is incorrect syntax.

table[ColumnName] returns a list with wrong type

Cause: All elements in the list reflect the column type, but if the column has type any (untyped), the list elements may be text instead of numbers or dates.

Solution: Always apply Table.TransformColumnTypes before extracting columns as lists. Type-safe columns produce correctly typed list elements.

Key Functions — Module 4 Quick Reference

All list functions introduced in this module, organised by category.

Function	Category	Key Behaviour
List.Count	Inspect	Number of elements
List.First	Access	First element (with optional default)
List.Last	Access	Last element (with optional default)
List.FirstN	Access	First n elements or elements while condition is true
List.LastN	Access	Last n elements
List.Range	Access	Sublist from offset with optional count
List.IsEmpty	Inspect	True if the list has zero elements
List.Contains	Membership	True if value is in list
List.ContainsAll	Membership	True if ALL values from a second list are in the list
List.ContainsAny	Membership	True if ANY value from a second list is in the list
List.PositionOf	Search	Zero-based index of value; -1 if not found
List.PositionOfAny	Search	Position of first occurrence of any value from a list
List.Min	Stat	Minimum value
List.Max	Stat	Maximum value
List.Median	Stat	Median (middle) value
List.Mode / List.Modes	Stat	Most frequent value(s) as a list
List.Percentile	Stat	Value at specified percentile (0.0-1.0)
List.Transform	Transform	Apply function to every element; returns new list
List.Select	Transform	Keep only elements where condition is true
List.Sort	Transform	Sort ascending, descending, or by custom comparer
List.Distinct	Transform	Remove duplicate values
List.Reverse	Transform	Reverse element order
List.Skip	Transform	Skip first n elements (or while condition is true)
List.RemoveFirstN	Transform	Remove first n elements
List.RemoveLastN	Transform	Remove last n elements
List.RemoveNulls	Transform	Remove all null values
List.RemoveItems	Transform	Remove all elements appearing in a second list
List.RemoveMatchingItems	Transform	Remove items with custom comparer
List.Repeat	Transform	Repeat the list n times
List.ReplaceValue	Transform	Replace occurrences of a value
List.Zip	Combine	Pair n-th elements from each input list

List.Sum	Aggregate	Sum of all values (nulls ignored)
List.Average	Aggregate	Arithmetic mean (nulls ignored)
List.StandardDeviation	Aggregate	Population standard deviation
List.Variance	Aggregate	Population variance
List.Covariance	Aggregate	Covariance of two equal-length lists
List.Correlation	Aggregate	Pearson correlation coefficient
List.Product	Aggregate	Product (multiplication) of all values
List.Accumulate	Advanced	Reduce/fold: iterate with running state (state, current) => newState
List.Union	Set	All distinct values across multiple lists
List.Intersect	Set	Values present in ALL input lists
List.Difference	Set	Values in list1 NOT in list2
List.Combine	Combine	Concatenate multiple lists (preserves duplicates)
List.ToTable	Convert	Convert list to single-column table
List.Numbers	Create	Arithmetic numeric sequence
List.Dates	Create	Date sequence (foundation of Date dimension table)
List.Times	Create	Time value sequence at regular intervals
List.Generate	Create/Advanced	Iterative sequence: initial / condition / next / selector
Record.FieldValues	Convert	Extract all field values from a record as a list
Record.FieldNames	Convert	Extract all field names from a record as a list
Record.FromList	Convert	Build a record from a values list and a names list
Table.ToList	Convert	Convert table rows to a list of combined values

TIP Complete formula reference for all 12 modules: [PowerQuery_M_Formula_Reference.pdf](#)

Module Summary

- A list is an ordered, zero-indexed sequence of values. It is the fundamental building block behind table columns, set operations, dynamic column selection, aggregation, and iterative computation in M.
- List.Transform applies a function to every element; List.Select filters elements by condition. These are the list equivalents of Table.TransformColumns and Table.SelectRows.
- Aggregation functions (List.Sum, List.Average, List.StandardDeviation, List.Percentile) return a single scalar from a list and work directly on table columns via table[Column].
- Set operations: List.Union (all distinct values), List.Intersect (common values), List.Difference (A minus B). All take a list of lists as input — note the outer braces.
- List.Accumulate is a reduce/fold operation. It processes each element with a running state — use it for concatenation, counting, and custom aggregations on small lists.

- `List.Generate` is a while loop that builds a list iteratively. It is the standard pattern for paginated API ingestion, controlled date sequence generation, and any scenario requiring iterative state progression.
- Seven professional patterns: dynamic column selection, inline lookup, multi-value parameter filtering, cross-join from two lists, running cumulative totals, schema validation, and paginated API ingestion — all driven by list functions.

Next Module Module 5: Record Functions — Covers all `Record.*` functions: creation, field access, transformation, merging, and dynamic record manipulation. Also introduces the M type system for records and the critical `Table.ExpandRecordColumn` pattern for nested JSON/API data.